IMPERIAL COLLEGE LONDON EE2-18

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2017

EEE PART II: MEng, BEng and ACGI

Corrected copy

ALGORITHMS AND DATA STRUCTURES

Tuesday, 8th May 2017, 10:00am

Time allowed: 1:30 hours

There are TWO questions on this paper.

Answer BOTH questions.

Question 1 carries 40% of the marks. Question 2 carries 60%.
This exam is OPEN BOOK

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible Marker: C. Bouganis

© Imperial College London

Special information for invigilators:
Students may bring any written or printed aids into the examination. The students are allowed to take a copy of the exam.

Information for candidates:
Marks may be deducted for answers that use unnecessarily complicated algorithms, or more than the minimum number of functions.

The Questions

1. a) Figure 1.1 shows a C++ function that calculates the value of the function described in equation (1.1), for any non-negative integer n.

$$f(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ f(n-1) + f(n-2) & n > 1 \end{cases} \quad (1.1)$$

Provide a recursive C++ function that performs the above calculation.

[6]

b) Identify up to six syntactic or functional errors in the C++ code shown in Figure 1.1.

```
void calculateF(N) {
    if (n==0)
        return 0;
    else {
        return 3*calculateFR(n);
        return 2;
    }
}
```

Figure 1.1 calculateF() function.

[6]

[continued on the following page]

c) i) A set of numbers is inserted in an ordered binary tree (ascending ordered tree). Draw a tree for the following set assuming that the elements in the set are inserted in the order shown, and state whether the resulted tree is balanced or not (justify your answer)

{5, 2, 8, 10}

[4]

ii) In the previous resulted tree, add the number 12. State whether the resulted tree is balanced or not, and justify your answer. If the tree is currently unbalanced, then balance it using single and/or double rotations and draw the resulting tree. Show all intermediate steps.

[4]

iii) What is the height of the tree resulting from part (ii)?

[2]

iv) An alternative data structure to a binary tree is a linked list. Consider an ordered (ascending order) linked list data structure, and draw the final structure that results from storing the set of data described in part (ii). Comparing the resulting data structure to the data structure described in part (ii), which one provides faster average access to the data and why?

[2]

d) Draw a parse tree for the following expressions, assuming the normal priorities of the operators:

i) $1 * 3/4$

[2]

ii) $2 + (4 + 3) * 5$

[2]

[continued on the following page]

e) Consider the C++ code segment in Figure 1.2. With justification, state the values of variables *x*, *y* at points A and B of the code. With justification, state whether this code segment has a memory leak or not.

```
    int x=5;
    int y=10;
    int *p1 = &x;
    int *p2 = &y;
    *p1 = *p1 + *p2;
    x = 2*x;
A
    p2 = p1;
    *p2 = 2*(*p1);
B
```

Figure 1.2 Code segment.

[5]

[continued on the following page]

f) Figure 1.3 shows the type declaration for a dynamic linked list, where each node stores an integer in the *data* field. An empty list is not a valid instance of the data structure.

```
struct Node {
    int data;
    Node * next;
};

typedef Node * NodePtr;
NodePtr hdList = NULL;
```

Figure 1.3 Linked list declaration.

i) Write a C++ function/procedure that takes a pointer to a linked list as input and returns the sum of all values stored in the *data* field of the elements in the list whose values are more or equal to 10. In the case of an empty list or where the list does not contain any such node, the function should return 0.

[3]

ii) Write a C++ function/procedure that takes as input a linked list and returns the pointer to the first node of the list whose stored value is more or equal to 10. In the case of an empty list or where such node does not exists, the function should return the *NULL* value.

[4]

2. Consider an ordered binary tree structure, where each node of the tree structure can store a positive integer number. Assume that an ascending ordering has been imposed to the tree. An instance of the binary tree structure is shown below.

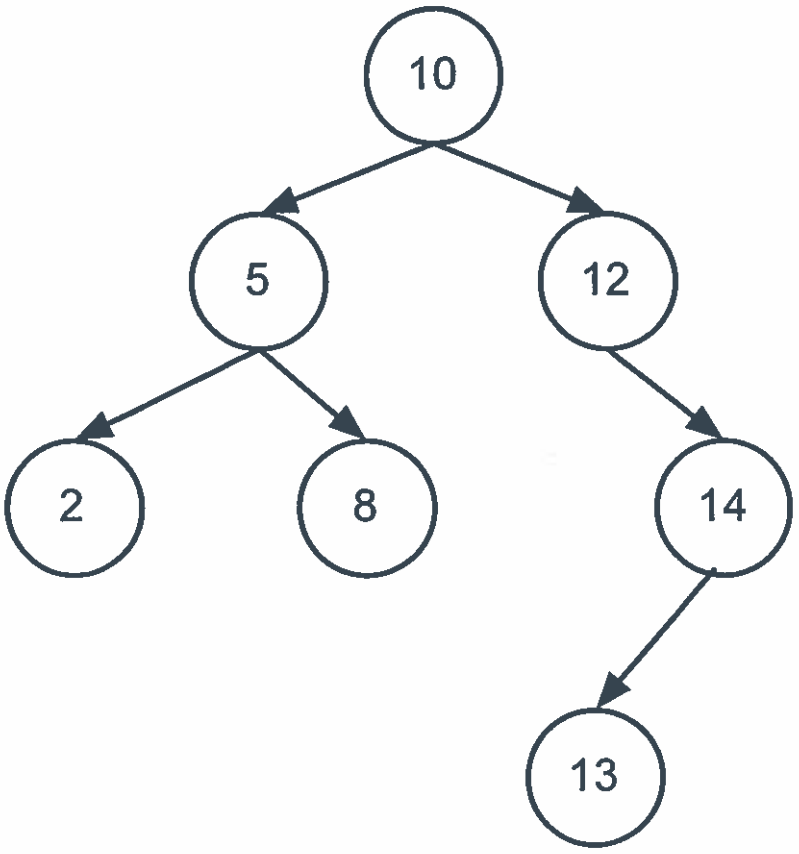


Figure 2.1 Tree structure.

a) Define a structure *treeNode* capable of representing a node of the tree.

[5]

b) Write a recursive function/procedure that takes as input the pointer to the root of the tree, and returns the number of nodes that have stored an even number. Show how your recursive function/procedure will be invoked. You can always pass more input arguments in your function/procedure.

[10]

c) Write a recursive function/procedure that takes as input the pointer to the root of the tree, and returns the number of nodes that have two children. Show how your recursive function/procedure will be invoked. You can always pass more input arguments in your function/procedure.

[10]

[continued on the following page]

d) Write a function/procedure that takes as input the pointer to the root of the tree and returns the pointer to the deepest internal node of the tree (i.e. the internal node with the maximum distance from the root of the tree). Show how your function/procedure will be invoked. You can always pass more input arguments in your function/procedure.

[15]

e) Write a recursive function/procedure that takes as input the pointer to the root of the tree, and returns *TRUE* if all the numbers stored in it's left and right subtrees are smaller or equal than the value stored in the root node, otherwise it returns *FALSE*. Show how your recursive function/procedure will be invoked. You can always pass more input arguments in your function/procedure.

[20]